

K-Means Clustering

```
In [ ]: K-Means is an algorithm that groups data into K clusters by minimizing the distance between data points and the
Unsupervised → No labels needed
Works best with numeric data
Need to choose K value manually
Sensitive to outliers.
#segment customers based on income and spending.
```

```
In [1]: import pandas as pd #used to handle dataset
import numpy as np #used for mathematical operations
import matplotlib.pyplot as plt #used to draw graphs
import seaborn as sns #to create informative graphs with less code
```

```
In [2]: df=pd.read_csv('Mall_Customers.csv') #it reads csv file into table
```

```
In [3]: df.head() #it shows top 5 rows
```

```
Out[3]:
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
In [4]: df.tail() # it shows bottom 5 rows
```

```
Out[4]:
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

```
In [5]: df.shape #to check no of rows and no of columns
```

```
Out[5]: (200, 5)
```

```
In [6]: df.info() #to check datatypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            200 non-null   int64
1   Genre                 200 non-null   object
2   Age                   200 non-null   int64
3   Annual Income (k$)    200 non-null   int64
4   Spending Score (1-100) 200 non-null   int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
In [7]: df.isnull().sum() #checking null values
```

```
Out[7]: CustomerID      0
Genre      0
Age        0
Annual Income (k$)    0
Spending Score (1-100) 0
dtype: int64
```

```
In [8]: df.duplicated().sum() #checking duplicate values
```

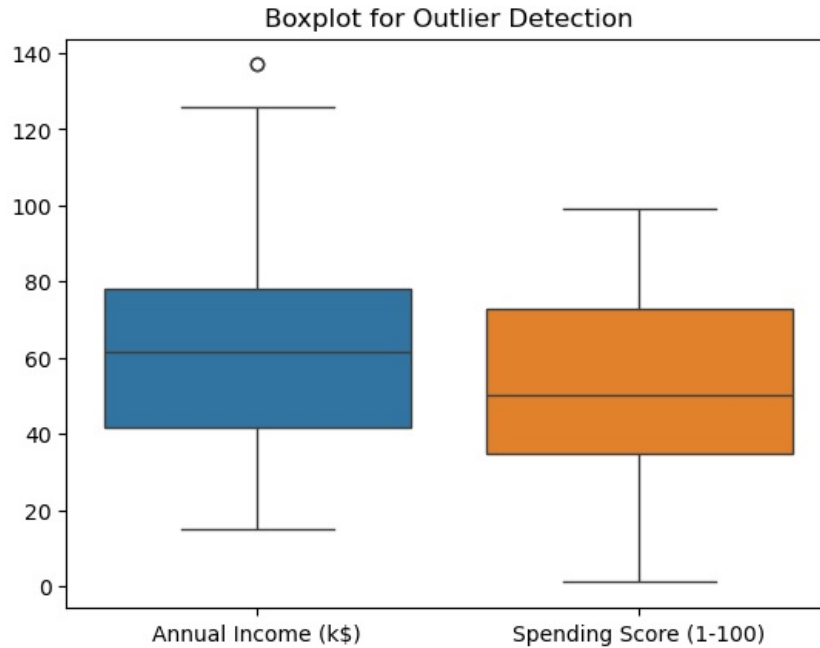
```
Out[8]: 0
```

```
In [9]: df.describe() #checking statistical measures
```

Out[9]:

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
In [10]: sns.boxplot(data=df[['Annual Income (k$)', 'Spending Score (1-100)']])#checking outliers for these 2 columns
plt.title("Boxplot for Outlier Detection")# it gives title to our graph
plt.show()#it will show the graph.
```



```
In [13]: X = df[['Annual Income (k$)', 'Spending Score (1-100)']] #selecting 2 columns for clustering.
# here x is input data for clustering
#we want to group customers based on income and spending behaviour
```

```
In [14]: from sklearn.cluster import KMeans # importing clustering algorithm
wcsc = [] # Within Cluster Sum of Squares(stores error[distance])

for i in range(1, 11):# try clusters from 1 to 10
    kmeans = KMeans(n_clusters=i, random_state=42)# no.of clusters=i
    kmeans.fit(X) #train model
    wcsc.append(kmeans.inertia_) #wcsc[error] (lower wcsc means better clustering)

# Plot
plt.plot(range(1, 11), wcsc, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
# inertia means total distance of all datapoints from their cluster[centroid].
# it tells how tightly data points group inside cluster
# points far -->high inertia, points close -->low inertia
# example the inertia values for different k-values fall from 1000000,70000,50000,45000,44448 ...
# inertia =(distance between point and its centroid)^2
# for each point distance to centroid,square it ,add all distances
```



```
y_kmeans = kmeans.fit_predict(X)
```

C:\Users\SRIKANTH\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
warnings.warn(

```
In [16]: df['Cluster'] = y_kmeans #to see no of clusters
df.head()
```

```
Out[16]:
```

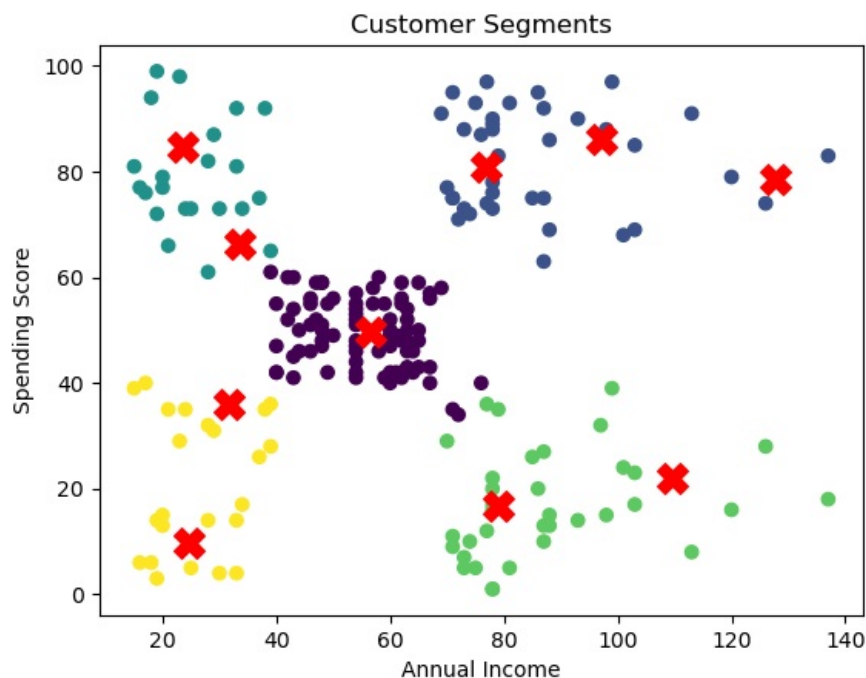
	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```
In [ ]: 0 -->med income and med spending
1 -->high income and low spending
2 -->low income and high spending
3 -->high income and high spending
4 -->low income and low spending
# cluster 4 means customer id no.1 belongs to "low income and low spending".
```

```
In [40]: plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=y_kmeans) #[:, 0]->income,[:, 1]->spending score c=y_kmeans->color

# Plot centroids
plt.scatter(kmeans.cluster_centers[:, 0], # cluster_centers means centroids
           kmeans.cluster_centers[:, 1],
           s=200, c='red', marker='X') #s means size(dot size) ,x->marker style

plt.title('Customer Segments')
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.show()
# here[:,0] means prints all rows of column1 i.e.,Income on x-axis
# here[:,1] means print all rows of column2 i.e., spending on y-axis
# centroid means average position of all points.
# if centroid size is 50,it is too small,if its 500 very big size
```



```
In [39]: from sklearn.metrics import silhouette_score #importing silhouette score for evaluating

score = silhouette_score(X, y_kmeans)
print("Silhouette Score:", score)
# how well each datapoint fits in its cluster compared to other clusters
#how close it is to its own cluster.
#calculate using s=b-a/max(a,b)
#a=distance to same cluster ,b=distance to nearest other cluster [distance calculate by using [Euclidean distance]]
```

Silhouette Score: 0.553931997444648

```
In [ ]: near 1-->good
```

```
near 0-->average
negative-->bad
```

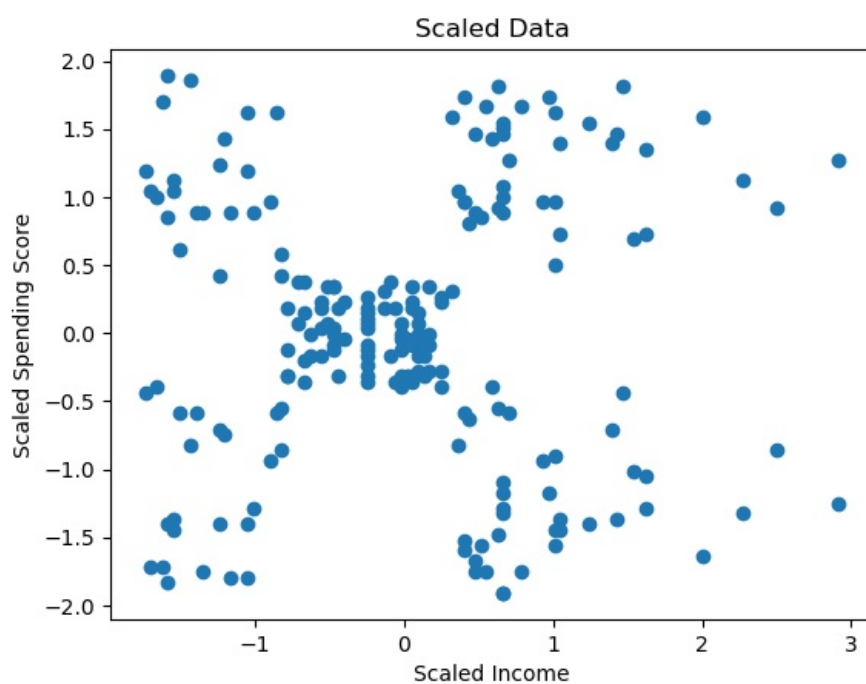
```
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
```

```
In [21]: df['Gender'] = df['Genre'].map({'Male': 0, 'Female': 1})
```

```
In [22]: from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [23]: plt.scatter(X_scaled[:, 0], X_scaled[:, 1])
plt.xlabel("Scaled Income")
plt.ylabel("Scaled Spending Score")
plt.title("Scaled Data")
plt.show()
```



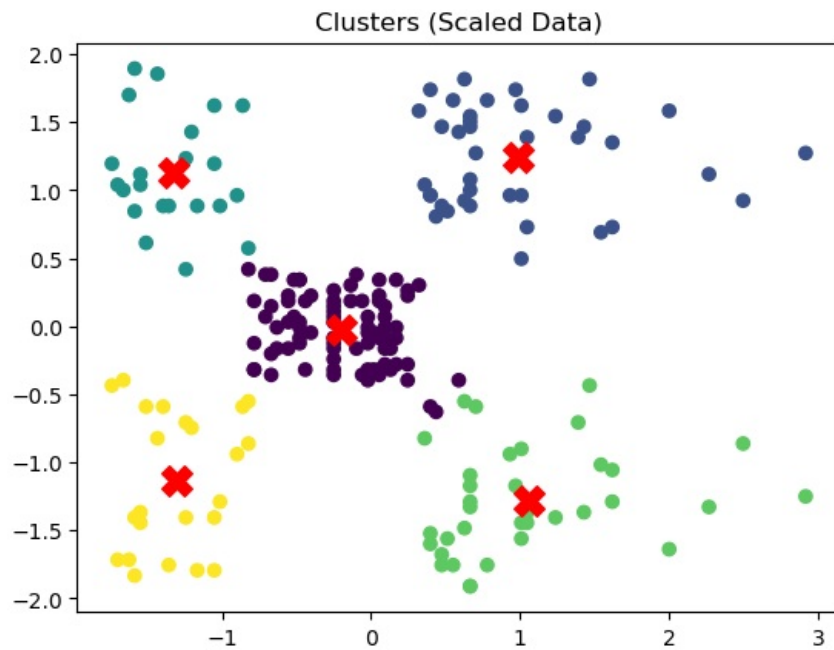
```
In [24]: kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

C:\Users\SRIKANTH\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
warnings.warn(

```
In [25]: plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans)

plt.scatter(kmeans.cluster_centers_[0],
           kmeans.cluster_centers_[1],
           s=200, c='red', marker='X')

plt.title("Clusters (Scaled Data)")
plt.show()
```



```
In [28]: from sklearn.metrics import silhouette_score

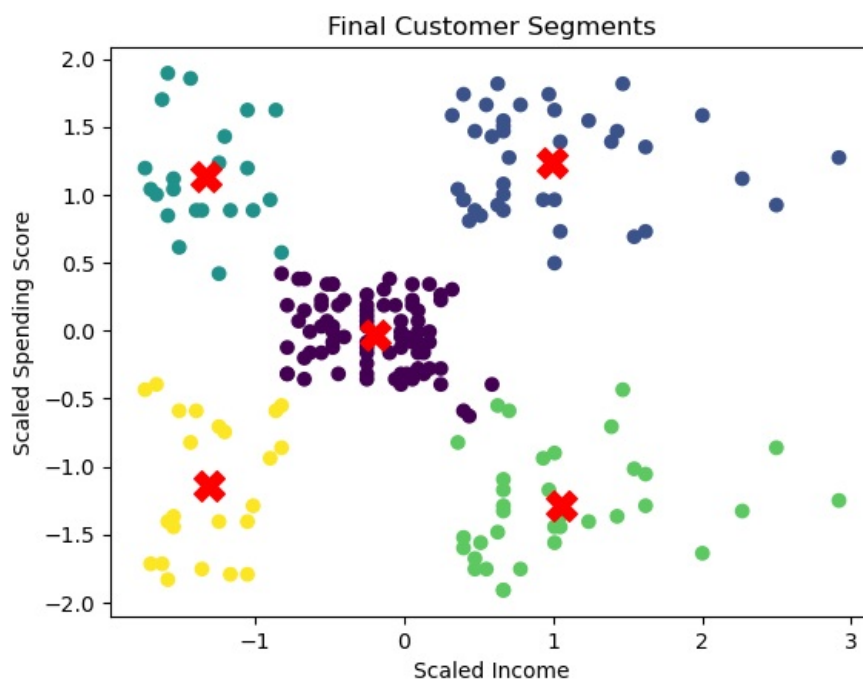
score = silhouette_score(X_scaled, y_kmeans)
print("Silhouette Score:", score)
```

Silhouette Score: 0.5546571631111091

```
In [29]: plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans)

plt.scatter(kmeans.cluster_centers_[0],
           kmeans.cluster_centers_[1],
           s=200, c='red', marker='X')

plt.title("Final Customer Segments")
plt.xlabel("Scaled Income")
plt.ylabel("Scaled Spending Score")
plt.show()
```



In []:

In []:

In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js